MAINCRAFTS TECHNOLOGY
www.maincrafts.com
hr@maincrafts.com

VLSI Design Internship - Task 4

RTL Design of Finite State Machines (FSM) and Control Units

Objective

In VLSI Design Internship - Task 3 (Task-3), you implemented sequential circuits such as flip-flops, registers, and counters, and learned how clock-driven systems behave.

In **Task-4**, you will advance toward **control-driven digital design**, which is a critical requirement in modern semiconductor systems.

The objective of this task is to:

- Understand **Finite State Machines (FSM)**
- Design **Moore and Mealy FSMs using Verilog**
- Implement a **traffic light controller**
- Design a **sequence detector (pattern recognition circuit)**
- Write testbenches and verify FSM transitions
- Analyze timing and state transitions through simulation

Why This Task Matters (Industry Context)

- FSMs are used in:
  - Processors (control unit)
  - Communication protocols
  - Embedded systems
  - ASIC/FPGA controllers
- Every RTL engineer must know:
  - State encoding
  - Timing control
  - Sequential decision logic

What You Will Build

You will design and simulate the following:

1. Moore FSM (basic state machine)
2. Mealy FSM (output dependent on input + state)
3. Traffic Light Controller (real-world system)
4. Sequence Detector (e.g., detect "1011" pattern)

Each design must include:

- Verilog RTL module
- Testbench
- Simulation waveform analysis

Core Concepts You Will Learn

Concept	Explanation
FSM	Circuit with defined states and transitions
Moore Machine	Output depends only on state
Mealy Machine	Output depends on state + input
State Encoding	Binary representation of states
State Transition	Movement between states on clock
Control Logic	Decision-making circuits

Tools & Software (Free)

Use the same tools from Task-3:

- EDA Playground (recommended)
- Icarus Verilog
- GTKWave
- ModelSim (optional)

Prerequisites

Before starting Task-4:

- Completion of Task-3
- Understanding of flip-flops and counters
- Knowledge of Verilog `always @(posedge clk)`
- Basic understanding of timing and simulation

Part A: Introduction to FSM

Finite State Machine consists of:

- States
- Inputs
- Outputs
- Transitions

Two types:

- Moore → Output = f(State)
- Mealy → Output = f(State, Input)

Part B: Moore FSM Design (Basic Example)

Problem Statement

Design a simple FSM that cycles through 3 states:
S0 → S1 → S2 → S0

Starter Verilog Code

```verilog
module moore_fsm(
    input clk,
    input reset,
    output reg [1:0] state
);

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;

always @(posedge clk or posedge reset) begin
    if (reset)
        state <= S0;
    else begin
        case(state)
            S0: state <= S1;
            S1: state <= S2;
            S2: state <= S0;
            default: state <= S0;
        endcase
    end
end

endmodule
```

Part C: Mealy FSM Design (Input-Based Output)

Problem Statement

Output becomes HIGH when input = 1 and current state is S1.

Starter Code

```verilog
module mealy_fsm(
    input clk,
    input reset,
    input in,
    output reg out
);

reg state;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 0;
        out <= 0;
    end else begin
        case(state)
            0: begin
                if (in) begin
                    state <= 1;
                    out <= 0;
                end
            end
            1: begin
                if (in)
                    out <= 1;
                else begin
                    state <= 0;
                    out <= 0;
                end
            end
        endcase
    end
end

endmodule
```

Part D: Traffic Light Controller (Industry-Level Mini Project)

System Behavior

State	Light
S0	Red
S1	Yellow
S2	Green

Starter Code

verilog

```verilog
module traffic_light(
    input clk,
    input reset,
    output reg [2:0] light
);

parameter RED = 3'b100, YELLOW = 3'b010, GREEN = 3'b001;

reg [1:0] state;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 0;
        light <= RED;
    end else begin
        case(state)
            0: begin state <= 1; light <= GREEN; end
            1: begin state <= 2; light <= YELLOW; end
            2: begin state <= 0; light <= RED; end
        endcase
    end
end

endmodule
```

Part E: Sequence Detector (Pattern Detection – 1011)

Objective

Detect binary sequence **1011** in serial input stream.

Starter Code

verilog

```verilog
module sequence_detector(
    input clk,
    input reset,
    input in,
    output reg detected
);
```

```
reg [2:0] state;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 0;
        detected <= 0;
    end else begin
        case(state)
            0: state <= (in) ? 1 : 0;
            1: state <= (in) ? 1 : 2;
            2: state <= (in) ? 3 : 0;
            3: begin
                if (in) begin
                    detected <= 1;
                    state <= 1;
                end else
                    state <= 2;
            end
        endcase
    end
end

endmodule
```

Part F: Testbench Development

Example:

verilog

```
module tb_fsm;

reg clk = 0;
reg reset = 1;

always #5 clk = ~clk;

initial begin
    #10 reset = 0;
    #100 $finish;
end

endmodule
```

Part G: Simulation Steps

1. Compile
   `iverilog fsm.v tb_fsm.v`
2. Run
   `vvp a.out`
3. View waveform
   `gtkwave dump.vcd`
4. Verify:
   - Correct state transitions
   - Output timing
   - Reset behavior

Expected Output

After completing Task-4, you should have:

- Working FSM-based Verilog designs
- Testbench simulations
- Waveform analysis showing state transitions
- Understanding of control logic design

Deliverables

Mandatory:

1. Verilog files:
   - `moore_fsm.v`
   - `mealy_fsm.v`
   - `traffic_light.v`
   - `sequence_detector.v`
2. Testbench files
3. Simulation screenshots
4. Short PDF including:
   - State diagrams
   - Explanation
   - Observations

---

1. Free Tools & Online Platforms

These tools are required to design, simulate, and verify FSM-based RTL circuits.

EDA Playground (Recommended – No Installation)

- Online Verilog/SystemVerilog simulator
- Best for quick testing and sharing code

Search topics:

- EDA Playground FSM Verilog tutorial
- How to simulate FSM in EDA Playground

Icarus Verilog (Open Source Simulator)

- Used for compiling and running Verilog code

Search topics:

- Icarus Verilog FSM example
- How to simulate Verilog FSM using Icarus

GTKWave (Waveform Viewer)

- Used to visualize state transitions and outputs

Search topics:

- GTKWave FSM waveform analysis
- How to debug FSM using GTKWave

ModelSim (Optional – Industry Tool)

- Advanced simulation used in real semiconductor companies

Search topics:

- ModelSim FSM simulation tutorial
- Verilog FSM waveform debugging ModelSim

---

2. FSM Fundamentals (Very Important)

Before coding, you must clearly understand how FSM works.

Search topics:

- Finite State Machine basics digital electronics
- Moore vs Mealy machine difference
- FSM state diagram explained
- Synchronous sequential circuits FSM
- State transition diagram tutorial

YouTube Learning Links

---

3. FSM Design in Verilog

Learn how FSM is implemented using Verilog HDL.

Search topics:

- FSM implementation in Verilog HDL
- Verilog state machine example
- always block FSM design posedge clk
- case statement FSM Verilog
- State encoding in Verilog

YouTube Learning Links

---

4. Moore vs Mealy Implementation

Understanding both types is critical for interviews and real projects.

Search topics:

- Moore machine Verilog example
- Mealy machine Verilog example
- Moore vs Mealy timing difference
- FSM output logic design

YouTube Learning Links

5. Traffic Light Controller (Real-World Project)

This is a very common **interview-level and industry-level FSM project.**

Search topics:

- Traffic light controller Verilog FSM
- Traffic signal design digital logic
- FSM real world examples Verilog

YouTube Learning Links

---

6. Sequence Detector Design (Pattern Detection)

Used in communication systems and digital pattern recognition.

Search topics:

- Sequence detector 1011 Verilog
- FSM pattern detection example
- Overlapping vs non-overlapping sequence detector

YouTube Learning Links

---

7. Testbench Development for FSM

Testbenches verify correct state transitions and outputs.

Search topics:

- Verilog testbench for FSM
- Clock generation in testbench
- FSM simulation example Verilog
- Writing stimulus in Verilog

YouTube Learning Links

---

8. Simulation & Waveform Analysis

You must verify behavior through waveforms.

Search topics:

- Verilog FSM simulation waveform
- GTKWave state transition analysis
- Debugging FSM timing issues
- Understanding digital waveforms

YouTube Learning Links

---

9. Advanced FSM Concepts (Optional but Recommended)

To move toward industry-level expertise:

Search topics:

- One-hot encoding FSM

- Binary vs Gray state encoding
- FSM optimization techniques
- Synchronous vs asynchronous FSM
- FSM design best practices

---

10. Free Academic Courses

For deeper theoretical understanding:

Search topics:

- NPTEL Digital System Design FSM
- NPTEL VLSI Design course
- FSM lecture digital electronics

---

11. Documentation & References

Search topics:

- Verilog FSM coding guidelines
- IEEE Verilog LRM FSM examples
- FSM design best practices PDF

---

12. Practice & Mini Projects

Practice is essential to master FSM design.

Search topics:

- Verilog FSM example projects
- FSM interview questions Verilog
- Digital design FSM problems

Try building:

- Vending machine FSM
- Elevator controller FSM
- Password lock system

---

13. Community Support (Doubt Solving)

Search topics:

- Stack Overflow FSM Verilog questions
- Reddit VLSI FSM discussion
- Verilog FSM debugging help

1. FSM Fundamentals (Start Here)

These cover:

- FSM basics
- Moore vs Mealy
- State diagrams
- Design flow

https://youtu.be/Z8d3Y96BDOo?si=YvkHe7tNIhgh1YEE
https://youtu.be/DgvTYv2OvBQ?si=crnuivoZdZ6Y4GcY

---

2. FSM Coding in Verilog

These help you:

- Convert logic → Verilog code
- Understand case statements
- Write synthesizable FSM

https://youtu.be/f7DSYhEGXFA?si=gxiEizPNwe3dCBNV
https://youtu.be/tzxaf-CNU3Q?si=2c-Bie2uCOXKMWYh

---

3. FSM from State Diagram

This is important for:

- Interview questions
- Real design workflow

https://youtu.be/O8Q09QMDVAQ?si=qhHXuI1vRyCJArPu

---

4. Sequence Detector (Important Project)

You will learn:

- 1011 pattern detection
- Overlapping vs non-overlapping
- Real FSM design logic

https://youtu.be/PbjntQf3sGc?si=vBunQq-eWV_mfdUL

5. Advanced FSM (Industry-Level Understanding)

These help you:

- Understand industry coding style
- Learn optimization techniques
- Prepare for VLSI interviews

https://youtu.be/kMuLqlrYs_A?si=QER3Fw7DUb5CMn3G
https://youtu.be/ZTKBu_C61Fg?si=a6fXfve2aLWE8DTv

---

6. Hands-on FPGA / Tool-Based FSM

These videos show:

- Real implementation
- Simulation + synthesis flow

https://youtu.be/gX6uY63fgHU?si=XjqsGWDxGexe0f4N
https://youtu.be/RvEE1mR4LbY?si=RxBtI7PFUgdvM07i

Final Suggestion (How to Use These Videos)

Follow this sequence:

1. Watch **FSM fundamentals videos**
2. Move to **Verilog coding tutorials**
3. Practice **sequence detector + traffic controller**
4. Use **advanced + FPGA videos** for deeper understanding